Ex. No.: 10
Date:

A PYTHON PROGRAM TO IMPLEMENT DIMENSIONALITY REDUCTION USING PCA

Aim:

To implement Dimensionality Reduction using PCA in a python program.

Algorithm:

Step 1: Import Libraries

Import necessary libraries, including pandas, numpy, matplotlib.pyplot, and sklearn.decomposition.PCA.

Step 2: Load the Dataset (iris dataset)

Load your dataset into a pandas DataFrame.

Step 3: Standardize the Data

Standardize the features of the dataset using StandardScaler from sklearn.preprocessing.

Step 4: Apply PCA

- Create an instance of PCA with the desired number of components.
- Fit PCA to the standardized data.
- Transform the data to its principal components using

transform. Step 5: Explained Variance Ratio

- Calculate the explained variance ratio for each principal component.
- Plot a scree plot to visualize the explained variance ratio.

Step 6: Choose the Number of Components

Based on the scree plot, choose the number of principal components that explain a significant amount of variance.

Step 7: Apply PCA with Chosen Components

Apply PCA again with the chosen number of components.

Step 8: Visualize the Reduced Data

- Transform the original data to the reduced dimension using the fitted PCA.
- Visualize the reduced data using a scatter plot.

Step 9: Interpretation

Interpret the results, considering the trade-offs between dimensionality reduction and information loss.

PROGRAM:

```
from sklearn import datasets
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import seaborn as sns
iris = datasets.load_iris()
df = pd.DataFrame(iris['data'], columns = iris['feature_names'])
df.head()
```

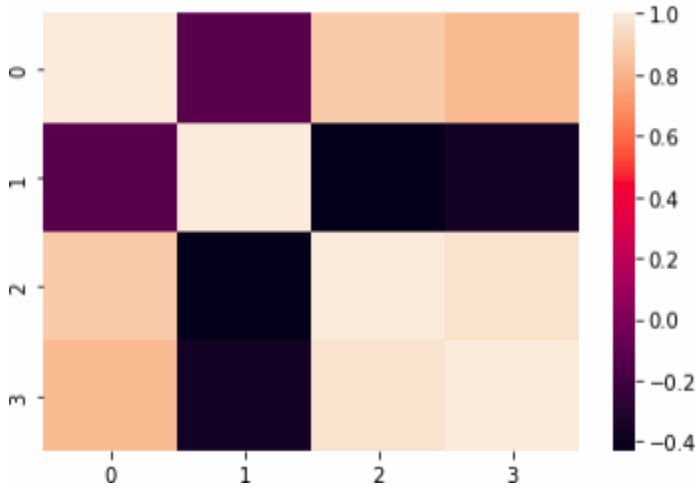
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```
scalar = StandardScaler()
scaled_data = pd.DataFrame(scalar.fit_transform(df)) #scaling the data
scaled_data
```

	0	1	2	3
0	-0.900681	1.019004	-1.340227	-1.315444
1	-1.143017	-0.131979	-1.340227	-1.315444
2	-1.385353	0.328414	-1.397064	-1.315444
3	-1.506521	0.098217	-1.283389	-1.315444
4	-1.021849	1.249201	-1.340227	-1.315444
...	...	...	...	...
145	1.038005	-0.131979	0.819596	1.448832
146	0.553333	-1.282963	0.705921	0.922303

```
sns.heatmap(scaled_data.corr())
```

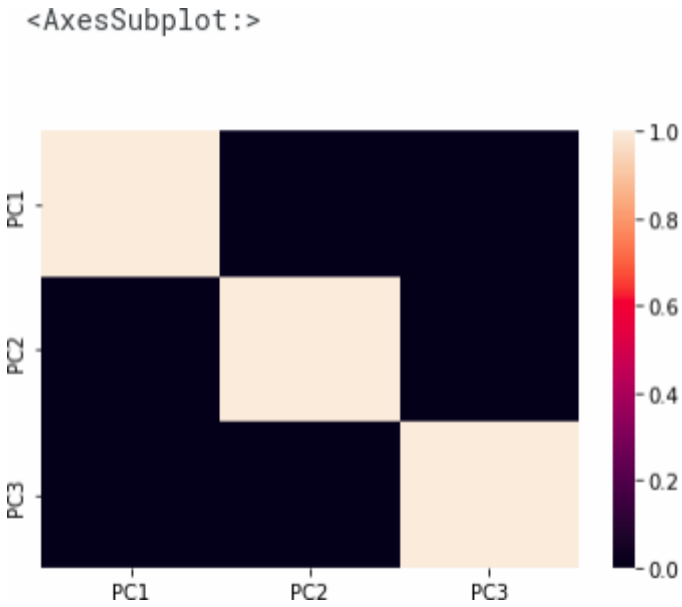
```
<AxesSubplot:>
```




```
pca = PCA(n_components = 3)
pca.fit(scaled_data)
data_pca = pca.transform(scaled_data)
data_pca = pd.DataFrame(data_pca,columns=['PC1','PC2','PC3'])
data_pca.head()
```

	PC1	PC2	PC3
0	-2.264703	0.480027	-0.127706
1	-2.080961	-0.674134	-0.234609
2	-2.364229	-0.341908	0.044201
3	-2.299384	-0.597395	0.091290
4	-2.389842	0.646835	0.015738

```
sns.heatmap(data_pca.corr())
```



RESULT:-

Thus Dimensionality Reduction has been implemented using PCA in a python program successfully and the results have been analyzed

VIVA QUESTIONS

1. What is the primary goal of univariate regression?

   To model the relationship between a single predictor variable and a response variable.

2. In bivariate regression, how many predictor variables are used?

   One predictor variable.

3. Which type of regression analysis would you use to analyze the relationship between multiple predictor variables and a single response variable?

   Multivariate regression.

4. What is the equation of a simple linear regression model?

   $$y = \beta_0 + \beta_1 x + \epsilon.$$

5. Which method is commonly used to estimate the coefficients in linear regression? Least squares method.

6. In multivariate regression, what does the term β_j represent?

   The coefficient for the jth predictor variable.

7. Which of the following is a key assumption of linear regression?

   Linearity, independence, homoscedasticity, and normality of residuals.

8. What is multicollinearity in the context of multivariate regression?

   A situation where predictor variables are highly correlated with each other.

9. Which statistical measure indicates the proportion of the variance in the dependent variable that is predictable from the independent variables?

   R-squared (R^2).

10. In the context of regression analysis, what is meant by the term 'overfitting'?

    Overfitting occurs when a model is too complex and captures the noise in the data rather than the underlying relationship.

11. What do the coefficients alpha and beta represent in the linear regression equation.

    $$y = \alpha + \beta x?$$

    In the linear regression equation, α (alpha) represents the y-intercept of the regression line, and β (beta) represents the slope of the regression line.

12. What is the main goal of linear regression?

    The main goal of linear regression is to find the best-fitting straight line through the data points that minimizes the sum of the squared differences (residuals) between the observed values and the values predicted by the line.

13. How is the slope (β) of the regression line calculated in the Least Squares Method?

    The slope (β) is calculated using the formula $\beta = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$, where x_i and y_i are the individual data points, and $\bar{x}$ and $\bar{y}$ are the means of x and y respectively.

14. What is the purpose of using the mean of x and y in the Least Squares Method?

    The mean of x and y is used to center the data around the origin, which simplifies the calculation of the slope and intercept and reduces numerical errors.

15. What is the purpose of the `np.mean()` function in the program?

    The `np.mean()` function is used to calculate the mean (average) of the array elements.

16. In the context of the program, what does the variable `alpha` represent? In the program, `alpha` represents the y-intercept of the regression line.

17. What is the purpose of logistic regression?

    The purpose of logistic regression is to model the probability of a binary outcome (0 or 1) based on one or more predictor variables. It is used for classification tasks.

18. What is the sigmoid function and why is it used in logistic regression?

    The sigmoid function is a mathematical function that maps any real-valued number into the range of 0 to 1. It is used in logistic regression to model the probability of the target variable being in a particular class.

19. What does the `predict_proba` method of the LogisticRegression class in `sklearn` return?

    The `predict_proba` method returns the probability estimates for each class. For binary classification, it returns a 2D array where each row contains the probabilities for the two classes (e.g., [probability of class 0, probability of class 1]).

20. How is the logistic regression model trained in the `sklearn` library?

    The logistic regression model is trained in `sklearn` using the `fit` method, which takes the training data (features and labels) and fits the model to it by optimizing the coefficients to best separate the classes.

21. What is the decision boundary in logistic regression?

    The decision boundary in logistic regression is the threshold at which the predicted probability is 0.5. At this point, the model switches from predicting class 0 to class 1 (or vice versa), effectively separating the two classes based on the predictor variables.

22. What is the main purpose of a perceptron?

    The main purpose of a perceptron is to classify input data into one of two classes by learning a linear decision boundary.

23. What is the role of the activation function in a perceptron?

The activation function in a perceptron determines the output of the perceptron based on the weighted sum of the inputs. It maps the input to an output of either 0 or 1, depending on whether the input exceeds a certain threshold.

Suggested Formula: $\text{Output} = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$

24. What is the role of hidden layers in a Multi-Layer Perceptron?

Hidden layers in a Multi-Layer Perceptron (MLP) allow the network to learn and represent complex patterns in the data by transforming the inputs through non-linear functions multiple times before producing the output.

25. How does backpropagation update the weights in a neural network?

Backpropagation updates the weights in a neural network by computing the gradient of the loss function with respect to each weight. It uses this gradient to adjust the weights in the direction that minimizes the loss, typically using an optimization algorithm like gradient descent.

26. What is the significance of the activation function in an MLP?

The activation function introduces non-linearity into the model, enabling the MLP to capture and learn complex patterns in the data. Without non-linear activation functions, the MLP would only be able to model linear relationships.

27. Why is it important to set a maximum number of iterations (`max_iter`) when training an MLP?

Setting a maximum number of iterations (`max_iter`) ensures that the training process stops even if the optimization algorithm has not converged. This prevents the model from running indefinitely and allows for control over the training time.

28. What is the purpose of the `random_state` parameter in the MLPRegressor?

The `random_state` parameter ensures reproducibility by controlling the random number generation used during the training process. This allows the same results to be obtained when the model is trained multiple times under the same conditions.

29. What is the purpose of using the `face_recognition` library in this program?

The `face_recognition` library is used for loading and handling face images, detecting faces within images, and extracting face encodings which are essential for training the SVM classifier.

30. What does the `train_test_split` function do in this program?

The `train_test_split` function splits the dataset into training and testing sets, ensuring that the model can be trained on one portion of the data and tested on another to evaluate its performance.

31. What kernel is used in the SVM classifier in this program?

    The SVM classifier uses the default RBF kernel, which is indicated by the parameter `gamma='scale'` in the `svm.SVC` initialization.

32. How are face encodings used in the context of this program?

    Face encodings are numerical representations of the facial features. These encodings are used as input features for the SVM classifier to distinguish between different faces.

33. What is the significance of the accuracy metric printed by the program?

    The accuracy metric indicates the percentage of correctly classified instances out of the total instances in the testing set. It gives a measure of how well the SVM classifier is performing in recognizing faces.

34. What is a Decision Tree in the context of machine learning?

    A Decision Tree is a supervised learning algorithm used for classification and regression tasks. It splits the data into subsets based on the value of input features, with each split represented as a branch, and the final outcome represented as a leaf node.

35. What is the primary objective of a Decision Tree algorithm?

36. The primary objective of a Decision Tree algorithm is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. It aims to minimize classification error or, in the case of regression, minimize prediction error.

37. How does a Decision Tree determine the best split at each node?

    A Decision Tree determines the best split at each node by evaluating various criteria such as Gini impurity, Information Gain (based on entropy), or Mean Squared Error (for regression). The split that results in the highest information gain or the lowest impurity is chosen.

38. What are some common hyperparameters used to control the growth of a Decision Tree? Common hyperparameters include `max_depth` (the maximum depth of the tree), `min_samples_split` (the minimum number of samples required to split an internal node), `min_samples_leaf` (the minimum number of samples required to be at a leaf node), and `max_features` (the number of features to consider when looking for the best split).

39. What are some methods to prevent overfitting in Decision Trees? Methods to prevent overfitting in Decision Trees include:

    Pruning: Removing parts of the tree that provide little power to classify instances.

    Setting Maximum Depth: Limiting the depth of the tree to prevent it from growing too complex.

    Minimum Samples for Split/Leaf: Setting a minimum number of samples required to split an internal node or to be at a leaf node.

    Cross-Validation: Using cross-validation to tune hyperparameters and assess the model's

performance on unseen data.

40. What does "Ada" in AdaBoost stand for?

"Ada" in AdaBoost stands for "Adaptive."

41. What is the main purpose of AdaBoost?

    The main purpose of AdaBoost is to combine multiple weak classifiers to form a strong classifier.

42. How does AdaBoost assign weights to misclassified instances?

    AdaBoost increases the weights of misclassified instances to focus on harder cases in subsequent iterations.

43. What is a weak learner in the context of AdaBoost?

    A weak learner is a classifier that performs slightly better than random guessing.

44. What type of models are typically used as weak learners in AdaBoost?

    Decision stumps (one-level decision trees) are typically used as weak learners in AdaBoost.

45. How does Gradient Boosting minimize the loss function?

    Gradient Boosting minimizes the loss function by sequentially adding models that fit the residual errors of the combined ensemble of previous models.

46. What is the main difference between AdaBoost and Gradient Boosting in terms of how they build models?

    The main difference is that AdaBoost focuses on adjusting weights of misclassified instances, while Gradient Boosting builds models to correct the residual errors of the preceding models.

47. What type of machine learning algorithm is K-Nearest Neighbors?

    K-Nearest Neighbors is a supervised learning algorithm.

48. What is the primary objective of the KNN algorithm?

    The primary objective of the KNN algorithm is to classify a data point based on how its neighbors are classified.

49. How is the 'K' in KNN defined?

    The 'K' in KNN is the number of nearest neighbors to consider when making a classification or prediction.

50. What distance metric is commonly used in KNN to find the nearest neighbors?\

    The Euclidean distance is commonly used in KNN to find the nearest neighbors.

51. What is the main disadvantage of the KNN algorithm?

    The main disadvantage of the KNN algorithm is its high computational cost, especially with large datasets, because it requires calculating the distance of each query point to all points in the training set.

52. Can KNN be used for regression problems?

Yes, KNN can be used for regression problems by averaging the values of the K nearest neighbors.

53. How does the choice of 'K' affect the KNN algorithm's performance?

    A small 'K' can lead to overfitting, while a large 'K' can lead to underfitting. The choice of 'K' needs to balance bias and variance.

54. What type of machine learning algorithm is K-Means?

    K-Means is an unsupervised learning algorithm.

55. What is the primary objective of the K-Means algorithm?

    The primary objective of the K-Means algorithm is to partition the data into K clusters, where each data point belongs to the cluster with the nearest mean.

56. How is the 'K' in K-Means defined?

    The 'K' in K-Means is the number of clusters to form in the data.

57. What is the first step in the K-Means clustering process?

    The first step in the K-Means clustering process is to randomly initialize K centroids.

58. How do you assign data points to clusters in K-Means?

    Data points are assigned to the cluster with the nearest centroid, based on the Euclidean distance.

59. What happens after data points are assigned to clusters in K-Means?

    After data points are assigned to clusters, the centroids are recalculated as the mean of all points in each cluster, and the process is repeated until convergence.

60. What is a common method to determine the optimal number of clusters in K-Means?

    A common method to determine the optimal number of clusters is the Elbow Method, which involves plotting the within-cluster sum of squares (WCSS) against the number of clusters and looking for an 'elbow' point where the rate of decrease sharply slows down.

61. What is Principal Component Analysis (PCA)?

    Principal Component Analysis (PCA) is a statistical technique used to reduce the dimensionality of a dataset while retaining most of the variation in the data.

62. How is a principal component defined in PCA?

    A principal component is a linear combination of the original variables that captures the maximum variance in the data.

63. What is the purpose of using PCA?

    The purpose of PCA is to simplify the complexity in high-dimensional data while retaining trends and patterns by transforming the data to a new set of variables

(principal components).

64. What does an eigenvalue represent in the context of PCA?

An eigenvalue represents the amount of variance captured by each principal component. Larger eigenvalues indicate that the principal component accounts for more variance in the data.

65. Why is it important to standardize the data before performing PCA?

    Standardizing the data is important because PCA is affected by the scale of the variables. Standardization ensures that each variable contributes equally to the analysis.

66. How do you determine the number of principal components to retain in PCA?

    The number of principal components to retain is typically determined by the cumulative explained variance, choosing enough components to capture a high percentage (e.g., 90-95%) of the total variance.

67. What are the advantages of using PCA for dimensionality reduction?

    The advantages of using PCA include reducing the dimensionality of the data, which simplifies analysis, reduces computational cost, and mitigates the risk of overfitting while retaining the essential patterns and relationships in the data.